

This is a hands-on lab on Knowledge Graph (KG) exploitation for analytical tasks. We will explore how to leverage the graph structure with Graph Neural Networks (GNNs) and Knowledge Graph Embeddings (KGEs), and how they can benefit from each other.

In the following, we provide the environment setup and then exercises to be solved. Each group must upload the solutions to Learn-SQL (look for the corresponding assignment event). One group member must submit the solutions (check below for a precise enumeration of what you need to submit) and list all group members in such document. Check the assignment deadline and be sure to meet it. It is a strict deadline!

What to deliver?

Upload ONE compressed file with your solutions. The main file of your solution must be a pdf file named as ‘[Group]-[MemberSurname]+.pdf’. The file must be structured according to the sections you will find below and, within each section, specify your solution following the instructions of each section. If you make any assumption not explicit in the statement, add a note in the corresponding section. For the questions requiring code you must attach a file per section named as follows: ‘[Group]-(SUB)SECTION-[MemberSurname]+’ or alternatively, a **CURATED** Google Colab notebook following the template provided. Reference them (or even include code snippets) in the pdf file.

Setup Instructions

For this assignment, a Google Colab notebook has been provided, containing the skeleton of the assignment as well as some basic helper functions (you can change/improve them if you want!). To set it up, follow the steps below:

- Create a copy of the Colab Notebook to your own Google Drive.
- Install the required GNN (PyTorch) and KGE (PyKEEN) libraries.
- In the Runtime settings, make sure to use one of the freely available GPUs.

Note: The Google Colab notebook has been provided to avoid dependency issues, ensure that everyone can start working smoothly and to speed up training with the GPUs provided. However, feel free to use your local machine/environment and refer to the notebook for extra guidance.

A Getting Familiar with KGEs

Objective

In the first part of this assignment, we are going to get familiar with KGEs. To do so, we are going to use the **PubMed** dataset, a citation dataset from the biomedical domain which contains 19717 papers (nodes) and 44338 citations links (edges). Therefore, this can be seen as a simple KG with one relation type: *cites*.

A.1 The most basic model

First of all, we will start generating embeddings with *TransE*, a very simple and interpretable model. Do not worry about obtaining the optimal results, a model with a sensible hyperparameter configuration will be enough, but get familiar with some of the capabilities provided by PyKEEN https://pykeen.readthedocs.io/en/stable/tutorial/first_steps.html.

Once you have the embeddings, choose a paper from the KG and compute the most probable embedding for a paper that cites works similar to the ones the paper you have chosen may cite. Provide a 2D sketch of your reasoning. Then retrieve the actual paper whose embedding lies closest to the vector you found.

A.2 Improving TransE

In the previous task we have used TransE as the base KGE model. This model provides good interpretability but has some drawbacks. Given the structure and data of our KG, give an example (i.e., a set of triples) of where the modeling assumptions of TransE fail, and explain a KGE model which solves this issue.

A.3 Training KGEs

Now, train different translational and semantic KGE models on the KG (we recommend at least 4 models) **and report and comment the Hits@1, Hits@3 and MRR results based on your interpretation of the models.** Keep your best performing embeddings, as we will need them later. **Also, explore and comment on the effects of the training parameters, both the expected and the observed,** for *embedding dimension*, *number of negative samples per positive sample* and the *loss margin*.

A.4 Negative Sampling

In PyKEEN, there exist two different strategies to generate negative samples¹ (Uniform Sampling and Bernoulli Sampling), which change the probability that they give to corrupt either the tail entity or the head entity in the triple. Given our KG, compute the corruption probabilities that they assign. Why do we obtain these results? Are we benefiting from the Bernoulli Sampling strategy?

A.5 Disconnected Paper

Imagine we accidentally leave a paper in our dataset that neither cites any paper nor has any citations. After training the KGEs, what will have happened to the embedding vector of this isolated paper? If we visualized this vector in 2D using PCA or any other dimensionality reduction technique, where do you expect it to be located?

B Exploiting the Global Graph Structure

Objective

In this second part of the assignment, we are going to explore how analytical tasks can benefit from data being stored following a graph structure. To do so, we are going to use some additional data that is present in the **PubMed** dataset:

- Each node has an associated **feature vector of dimension 500**, which is a TF-IDF weighted **word vector of the abstract** of the paper.²
- Each node is associated with one of 3 classes, corresponding to **different types of diabetes**: Diabetes Mellitus Experimental, Diabetes Mellitus Type 1, Diabetes Mellitus Type 2.

This dataset has been designed to explore **semi-supervised learning**. That is, **only a small portion of the nodes' labels (i.e., the diabetes type) can be seen during training**. Therefore, we would like the graph structure to propagate the label information.

The objective of this second part will be to **create a classifier to associate each paper with the corresponding diabetes type**, and to explore how to exploit the resulting embeddings.

¹https://pykeen.readthedocs.io/en/stable/reference/negative_sampling.html

²<https://en.wikipedia.org/wiki/Tf-idf>

B.1 Let's forget about the graph

First of all, we will forget about the graph structure associated with the dataset, and we will consider only the feature vectors of the training nodes to train a classifier. For simplicity, we will focus on out-of-the-box classifiers, such as Random Forests (RF), Support Vector Machines (SVM) or Logistic Regression (LR), and we encourage you to use some AutoML tools to facilitate your work. **You must report in a table the obtained accuracy score over the test set.**

B.2 Exploiting the graph structure

Now, we are going to use the citation information to exploit the graph structure. To do so, we will create a GNN by using the **PyTorch** library, which provides an easy way to build Deep Learning (DL) architectures by changing different modules. In the notebook, we provided the general training workflow, and you will need to **modify the GNN class to create and test different GNN architectures**.

For this task, we expect you to use four different types of Neural Network (NN) layers:

- **Graph Neural Networks:** These layers will perform aggregation and combination of the information from the neighbor nodes. Therefore, when defining them we need to specify the input and the output sizes, and when using them we need to pass the node features (which will be of the input size) and the edge information. You can use the GNNs available in PyTorch³, which contain the ones seen in the lectures.
- **Linear Layer:** This layer will be used to transform an input vector of size n to a vector whose size equals the number of classes. It will be the last layer in our architecture.⁴
- **Activation Layers:** Activation layers are used to introduce non-linear behavior in the NN. They are placed after the intermediate layers, and only change the input values, not the size of the vectors.⁵
- **Dropout Layers:** These layers are placed after the intermediate layer's activation function. It randomly deactivates some of the values of the vector, stimulating the network to generalize and not overfit.⁶

A simple example of a GNN architecture can be seen below:

```
class GNN(nn.Module):  
    def __init__(self, in_channels, hidden_channels, out_channels):
```

³<https://pytorch-geometric.readthedocs.io/en/2.7.0/modules/nn.html#convolutional-layers>

⁴<https://docs.pytorch.org/docs/stable/generated/torch.nn.Linear.html>

⁵<https://docs.pytorch.org/docs/stable/nn.html#non-linear-activations-weighted-sum-nonlinearity>

⁶<https://docs.pytorch.org/docs/stable/nn.html#dropout-layers>

```

super().__init__()
# First, we define our building blocks
# GNN layer
self.conv1 = GCNConv(in_channels, hidden_channels)
# Final classifier layer (fully connected)
self.classifier = nn.Linear(hidden_channels, out_channels)

# Then, we combine them to create the architecture:
def forward(self, x, edge_index):
    # GCN layer
    x = self.conv1(x, edge_index)
    x = F.relu(x)
    x = F.dropout(x, p=0.5, training=self.training)
    # Classification Layer
    x = self.classifier(x)

    return x

```

In this task, you will create your own GNN architecture to solve the classification task. At least, you should consider one architecture consisting of **two GNN blocks** and a **final classification layer**, where the dimension of the vector before the linear classification layer is **256**. Feel free to optionally explore additional Deep Learning architectures to improve your results. **In the deliverable, report your proposed architecture(s) and the accuracy results obtained. Comment what you have observed.**

B.3 Tricking the GNN

Imagine you want to trick your trained GNN into misclassifying a specific 'Diabetes Mellitus Type 1' paper as 'Type 2'. You cannot alter the paper's TF-IDF features, but you are allowed to add 3 fake citation edges to the graph. Detail your strategy for selecting the nodes to connect to maximize the disruption to the model's prediction.

B.4 The more the merrier?

In task B.2, we created a DL architecture with two GNN layers. Now, we will explore what happens if we increase this number. **Create new GNN models with 2, 4, 8 and 16 GNN layers and report the accuracy results. What do you observe? Why do you think this is happening?**

B.5 What if we do not have initial information?

B.5.1 Random Initialization

In the graph we have been using for this assignment, each node has a feature vector associated with it. However, this might not be the case in a lot of scenarios in which we only have the graph structure. If that is the case, we cannot use the common ML algorithms we used in Task B.1, as they all need meaningful feature vectors as input. However, we could use a GNN model with random initialization, hoping the information from the edges is enough to obtain a meaningful representation. To investigate that, **use the same GNN for which you previously obtained the best-performing model and report the results when using random initialization.**

B.5.2 KGE Initialization

However, a common practice in GNNs is to initialize the vectors with the results of a KGE model applied to them. Typically, TransE is used as one of the most simple and less computationally demanding models. Now, use the resulting embeddings you obtained in task A.3 as the initial vectors to train the same GNN architecture you used with random initialization. **Report and comment the results.**

B.6 Link Prediction with GNNs

In the previous tasks, you implemented a GNN for node classification, where the model outputs a label for each node. Now suppose we want to create a GNN architecture for link prediction (i.e., predicting whether a citation exists between two papers). How would you approach that? What parts of the pipeline must be changed? What new architecture component(s) must be introduced? Provide a sketch of the architecture.

B.7 Where are the embeddings?

As a result of task B.2, we have created a classifier for the articles in the **PubMed** graph. However, at the same time we have indirectly obtained one embedding for each of them. In this task you should:

1. Choose your best performing GNN model.
2. Locate at which step in the architecture the embeddings are obtained.
3. Modify your forward function in your GNN class to return the embeddings. *Clue: You should return the embeddings after they are produced and without continuing with the pipeline, following this template:*

```
def forward(self, x, edge_index, return_embs=False)
# First part of the architecture
###
    if return_embs:
        return x
# Last part of the architecture
###
return x
```

4. Retrain the model with the modified GNN class.
5. Retrieve your embeddings by doing:

```
gnn_model.eval()
embs = gnn_model(data.x, data.edge_index, return_embs=True)
embs = embs.detach().cpu().numpy()
```

6. You can use these embeddings to perform any Machine Learning task. For this assignment, **you should perform dimensionality reduction (e.g., PCA) and visualize the result by coloring each embedding with its corresponding class.**

B.8 Embedding spaces

Now, you have two different sets of embeddings for the PubMed papers, the ones you obtained with KGEs and the ones obtained by GNNs. Answer the following questions:

- Are these embeddings the same? Would it be correct to create a new embedding space by randomly choosing between the embeddings from KGE and GNN for each paper?
- For a fixed paper P, retrieve its top-10 nearest neighbors by cosine similarity in both embedding spaces. Do you expect the two sets of neighbors to overlap significantly? Why or why not?

B.9 Generating embeddings for new papers

Suppose that you are using the generated embeddings (either the ones obtained with GNNs or with KGEs) for a new downstream task (e.g., recommending papers). A brand-new paper is published to PubMed today, and we immediately receive its TF-IDF feature vector and a list of 5 older papers it cites. Explain and compare the required steps to generate an embedding for this new paper using your trained KGE model versus your trained GNN model.